

Object Oriented Programming

Project Documentation

Submitted by:

Nigah Fatima

Roll Number:

25011519-002

Course Code:

AI-103

Submitted To:

Dr. Abd ur Rehman

Degree:

BS Artificial Intelligence

Department:

Computer Science

UNIVERSITY OF GUJRAT



A WORLD CLASS UNIVERSITY

1. Project Overview:

i. Project Title:

Smart Electricity Bill Manager

ii. Short Description:

The Smart Electricity Bill Manager is a desktop GUI application developed in Java using the Swing framework. It is designed for Pakistani households to track, calculate and manage their monthly electricity consumption and bills. The system allows users to add home appliances, monitor energy usage per room, set monthly budgets and export detailed PDF reports. An admin role is also included to manage the electricity rate.

iii. Main functionality already implemented in GUI:

- User Registration and Login with SHA-256 password hashing.
- Dark-themed GUI with a professional blue color scheme.
- **Manual Mode:** Add appliances (name, wattage, hours/day, room, type) and calculate monthly cost.
- **Three appliance types:** Standard, Heavy (+15% cost), and Intelligent (-15% cost).
- **Target Mode:** Set monthly budget and get optimization recommendations.
- **Analytics Tab:** View cost breakdown by room and by appliance type.
- **Bill History Tab:** Save and view monthly electricity bills over time.
- **Tips & Advice Tab:** Pakistan-specific energy saving tips.
- **Admin Panel:** Update electricity rate (Rs/kWh) — visible to admin only.
- **Settings:** Backup data, Restore data, Export full PDF report, Clear all appliances.
- **Session Timer:** Live clock showing how long the user has been logged in.
- **Data Persistence:** All data saved to .txt files and reloaded on next login.

iv. How the system works:

When the application starts, the main method in SmartElectricityManager sets up a dark theme and opens the LoginFrame. The user logs in or registers. Authentication is handled by AuthService, which reads and writes user data to text files. Once logged in, the DashboardFrame opens with multiple tabs. Each tab provides a specific feature. When the user adds an appliance, it is stored in the User object's appliance list and saved to disk. All

cost calculations are done by calling calculateMonthlyCost() on each Appliance object. On logout, data is saved again and the login screen reappears.

2. Class Design:

The project contains 10 classes organized into three layers: Entity classes (data), Service/Controller class (logic), and GUI classes (interface). Each class is described below.

i. User (Entity Class):

Purpose: Represents a registered user of the system. Stores all user-related data.

Attribute	Data Type	Access	Description
username	String	private	Unique login name.
passwordHash	String	private	SHA-256 hashed password(no plain text).
role	String	private	“admin” or “user”.
budget	double	private	Monthly electricity budget in Rupees.
appliances	ArrayList<Appliance>	private	List of appliances added by this user.
billHistory	ArrayList<BillHistory>	private	List of saved monthly bills.

Method	Return type	Description
User(username,passwordHash,role)	-	Constructor: initializes all fields, default budget Rs.10,000
getUsername()	String	Returns the username
getPasswordHash()	String	Returns the hashed password
getRole()	String	Returns the user role
getBudget()	double	Returns the user role
setBudget(b)	void	Updates monthly budget
getAppliances()	ArrayList	Returns list of appliances
getBillHistory()	ArrayList	Returns list of bill records

```

1 // Source code is decompiled from a .class file using FernFlower decompiler (from Intel)
2 import java.util.ArrayList;
3
4 class User {
5     private String username;
6     private String passwordHash;
7     private String role;
8     private double budget;
9     private ArrayList<Appliance> appliances;
10    private ArrayList<BillHistory> billHistory;
11
12    public User(String var1, String var2, String var3) {
13        this.username = var1;
14        this.passwordHash = var2;
15        this.role = var3;
16        this.budget = (double)10000.0F;
17        this.appliances = new ArrayList();
18        this.billHistory = new ArrayList();
19    }
20
21    public String getUsername() {
22        return this.username;
23    }
24
25    public String getPasswordHash() {
26        return this.passwordHash;
27    }
28
29    public String getRole() {
30        return this.role;
31    }
32
33    public double getBudget() {
34        return this.budget;
35    }
36
37    public void setBudget(double var1) {
38        this.budget = var1;
39    }

```

ii. BillHistory (Entity Class):

Purpose: Records a snapshot of the total electricity bill for a specific month.

Attribute	Data type	Access	Description
date	LocalDateTime	private	Exact date and time when record was saved
totalBill	double	private	Total calculated bill amount in Rupees
month	String	private	Month label e.g “May 2025” (formatted from data)

Method	Return type	Description
BillHistory(totalBill)	-	Constructor: auto-captures current date and formats month
getMonth()	String	Returns the month string e.g ‘May 2025’
getTotalBill()	Double	Returns the saved bill amount

```

1 // Source code is decompiled from a .class file using FernFlower decompiler (from Intel)
2 import java.time.LocalDateTime;
3 import java.time.format.DateTimeFormatter;
4
5 class BillHistory {
6     private LocalDateTime date = LocalDateTime.now();
7     private double totalBill;
8     private double rate;
9     private String month;
10
11     public BillHistory(double var1, double var3) {
12         this.totalBill = var1;
13         this.rate = var3;
14         this.month = this.date.format(DateTimeFormatter.ofPattern("MMMM yyyy"));
15     }
16
17     public String getMonth() {
18         return this.month;
19     }
20
21     public double getTotalBill() {
22         return this.totalBill;
23     }
24
25     public LocalDateTime getDate() {
26         return this.date;
27     }
28 }
29

```

iii. Appliance (Abstract Parent Class):

Purpose: Serves as the abstract base for all appliance types. Defines shared attributes and enforces that every subclass must implement its own cost calculation. This demonstrates **Abstraction** and **Inheritance**.

Attribute	Data type	Access	Description
name	String	protected	Appliance name e.g. 'Air Conditioner'
room	String	protected	Room where appliance is located
type	String	protected	Type label: Standard / Heavy / Intelligent
wattage	double	protected	Power consumption in Watts
hoursPerDay	int	protected	Daily usage hours
isEssential	boolean	protected	Whether appliance is marked as essential

Method	Return type	Description
Appliance(name, wattage, hoursPerDay, room, type)	Constructor	Initializes all shared fields
calculateMonthlyCost(rate)	abstract double	Must be overridden — calculates monthly electricity cost
getName()	String	Returns appliance name

getWattage()	double	Returns wattage
getHoursPerDay()	int	Returns daily hours
getRoom()	String	Returns room name
getType()	String	Returns type label
isEssential()	boolean	Returns whether it is essential
setHoursPerDay()	void	Updates daily hours
SetEssential()	void	Sets the essential flag

```

2  abstract class Appliance {
3      protected String name;
4      protected double wattage;
5      protected int hoursPerDay;
6      protected String room;
7      protected String type;
8      protected boolean isEssential;
9
10     public Appliance(String var1, double var2, int var4, String var5, String var6) {
11         this.name = var1;
12         this.wattage = var2;
13         this.hoursPerDay = var4;
14         this.room = var5;
15         this.type = var6;
16         this.isEssential = false;
17     }
18
19     public abstract double calculateMonthlyCost(double var1);
20
21     public String getName() {
22         return this.name;
23     }
24
25     public double getWattage() {
26         return this.wattage;
27     }
28
29     public int getHoursPerDay() {
30         return this.hoursPerDay;
31     }
32
33     public String getRoom() {
34         return this.room;
35     }
36
37     public String getType() {
38         return this.type;
39     }
40
41     public boolean isEssential() {

```

iv. StandardAppliance (Subclass of Appliance):

Purpose: Represents normal household appliances like fans, TVs, or light bulbs. Uses the standard energy formula without any multiplier.

Cost Formula:

Monthly Cost = (Wattage x Hours/Day / 1000) x 30 x Rate

Method	Return type	Description
StandardAppliance(name, wattage, hoursPerDay, room)	-	Constructor: sets type to 'Standard'
calculateMonthlyCost(rate)	double	Returns standard formula result

```

class StandardAppliance extends Appliance {
    public StandardAppliance(String var1, double var2, int var4, String var5) {
        super(var1, var2, var4, var5, "Standard");
    }

    public double calculateMonthlyCost(double var1) {
        return this.wattage * (double)this.hoursPerDay / (double)1000.0F * (double)30.0F * var1;
    }
}

```

v. HeavyAppliance (Subclass of Appliance):

Purpose: Represents power-intensive appliances like air conditioners, geysers, or electric heaters. A 15% extra cost is added to reflect higher energy consumption patterns.

Cost Formula:

Monthly Cost = (Wattage x Hours/Day x 1.15 / 1000) x 30 x Rate

```

class HeavyAppliance extends Appliance {
    public HeavyAppliance(String var1, double var2, int var4, String var5) {
        super(var1, var2, var4, var5, "Heavy");
    }

    public double calculateMonthlyCost(double var1) {
        return this.wattage * (double)this.hoursPerDay * 1.15 / (double)1000.0F * (double)30.0F * var1;
    }
}

```

Methods	Return type	Description
HeavyAppliance(name, wattage, hoursPerDay, room)	-	Constructor: sets type to 'Heavy'
calculateMonthlyCost(rate)	double	Returns formula with 1.15 multiplier (+15%)

vi. IntelligentAppliance (Subclass of Appliance):

Purpose: Represents smart or inverter-based appliances that are energy efficient. A 15% discount is applied to reflect reduced electricity usage.

Cost Formula:

Monthly Cost = (Wattage x Hours/Day x 0.85 / 1000) x 30 x Rate

Method	Return type	Description
IntelligentAppliance(name, wattage, hoursPerDay, room)	-	Constructor: sets type to 'Intelligent'
calculateMonthlyCost(rate)	double	Returns formula with 0.85 multiplier (-15%)

```
class IntelligentAppliance extends Appliance {
    public IntelligentAppliance(String var1, double var2, int var4, String var5) {
        super(var1, var2, var4, var5, "Intelligent");
    }

    public double calculateMonthlyCost(double var1) {
        return this.wattage * (double)this.hoursPerDay * 0.85 / (double)1000.0F * (double)30.0F *
    }
}
```

vii. AuthService (Service / Controller Class):

Purpose: The central controller of the application. Handles all business logic including authentication, data persistence (file read/write), rate management, and backup/restore operations. All GUI classes interact with this class.

Attribute	Data type	Description
users	ArrayList<User>	Stores all registered users in memory
rate	Double	Current electricity rate in Rs/kWh (default Rs.35)
DATA_DIR	String (final)	Folder path 'electricity_data' where all files are saved

Method	Type	Description
AuthService()	-	Constructor: creates data folder, loads all data, creates default admin

hashPassword(pwd)	String	Private: converts password to SHA-256 hash
findUser(username)	User	Private helper: linear search in ArrayList
userExists	boolean	Private helper: checks if username already registered
register(user, pwd, role)	boolean	Adds new user; returns false if username already exists
login(user, pwd)	User	Returns User object if credentials match, else null
getRate()	double	Returns current electricity rate
setRate()	void	Updates rate and saves to rate.txt
updateBudget(user,budget)	void	Updates a user's budget and saves
isAdmin(u)	boolean	Returns true if user has 'admin' role
saveUserData(user)	void	Writes appliances and bill history to .txt files
loadUserData(user)	void	Reads appliances and bill history from .txt files
saveAllData()	void	Private: writes users.txt and rate.txt
loadAllData()	void	Private: reads users.txt and rate.txt on startup
backupData(destPath)	void	Copies entire data folder to a backup location
restoreData(srcPath)	void	Restores data folder from backup
deleteDirectory(file)	void	Private recursive delete helper

```

import java.io.*;
import java.nio.file.*;
import java.security.MessageDigest;
import java.util.ArrayList;

class AuthService {
    private ArrayList<User> users = new ArrayList<>();
    private double rate = 35.0;
    @SuppressWarnings("unused")
    private final String DATA_DIR = "electricity_data";

    public AuthService() {
        new File("electricity_data").mkdirs();
        loadAllData();
        if (findUser(username: "admin") == null) {
            register(username: "admin", password: "admin123", role: "admin");
        }
    }

    // — helper: find user by username (replaces HashMap.get()) —————
    private User findUser(String username) {
        for (User u : users) {
            if (u.getUsername().equals(username)) {
                return u;
            }
        }
        return null; // not found
    }

    // — helper: check if username exists (replaces HashMap.containsKey()) —
    private boolean userExists(String username) {
        return findUser(username) != null;
    }

    // — password hashing —————
    private String hashPassword(String password) {
        try {
            MessageDigest md = MessageDigest.getInstance("SHA-256");

```

viii. LoginFrame (GUI Class):

Purpose: The first window shown when the application starts. Provides login and registration functionality. Holds a reference to AuthService to perform authentication. On successful login, opens DashboardFrame and closes itself.

Method/Field	Type	Description
auth	AuthService	Reference to backend service class
usernameField	TextField	Input field for username
passwordField	JPasswordField	Input field for password (hidden characters)
statusLabel	JLabel	Displays success or error messages
LoginFrame()	Constructor	Sets up window size, title, and calls initUI()
initUI()	void	Builds all GUI components: header, form, buttons
login()	void	Reads fields, calls auth.login(), opens DashboardFrame
register()	void	Reads fields, calls auth.register() with role='user'
styleTextField(tf)	void	Applies dark blue styling to text fields
createBrightButton	JButton	Helper: creates a styled button with hover effect

```

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Component;
import java.awt.Cursor;
import java.awt.FlowLayout;
import java.awt.Font;
import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;
import java.awt.Insets;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JPasswordField;
import javax.swing.JTextField;

class LoginFrame extends JFrame {
    private AuthService auth = new AuthService();
    private JTextField usernameField;
    private JPasswordField passwordField;
    private JLabel statusLabel;

    public LoginFrame() {
        this.setTitle("< Smart Electricity Bill Manager");
        this.setSize(520, 480);
        this.setDefaultCloseOperation(3);
        this.setLocationRelativeTo((Component)null);
        this.initUI();
    }

    private void initUI() {
        JPanel var1 = new JPanel(new BorderLayout());
        var1.setBackground(new Color(18, 25, 45));
        var1.setBorder(BorderFactory.createEmptyBorder(30, 30, 30, 30));
        JPanel var2 = new JPanel();
    }

```

ix. **DashboardFrame (GUI Class-Main Window):**

Purpose: The main application window shown after login. Contains a tabbed panel with all features. Interacts with both the User object and AuthService to read/write data.

Field / Method	Type	Description
currentUser	User	The logged-in user object
auth	AuthService	Backend service reference
tabbedPane	JTabbedPane	Holds all feature tabs
timerLabel	JLabel	Shows live session time (HH:MM:SS)
budgetLabel	JLabel	Shows current budget in top bar
rateLabel	JLabel	Shows current electricity rate in top bar
applianceListModel	DefaultListModel	Data model for the appliance JList
createManualPanel()	JPanel	Builds the 'Add Appliance' tab
createTargetPanel()	JPanel	Builds the 'Budget & Optimize' tab

x. SmartElectricityManager (Main Entry Class):

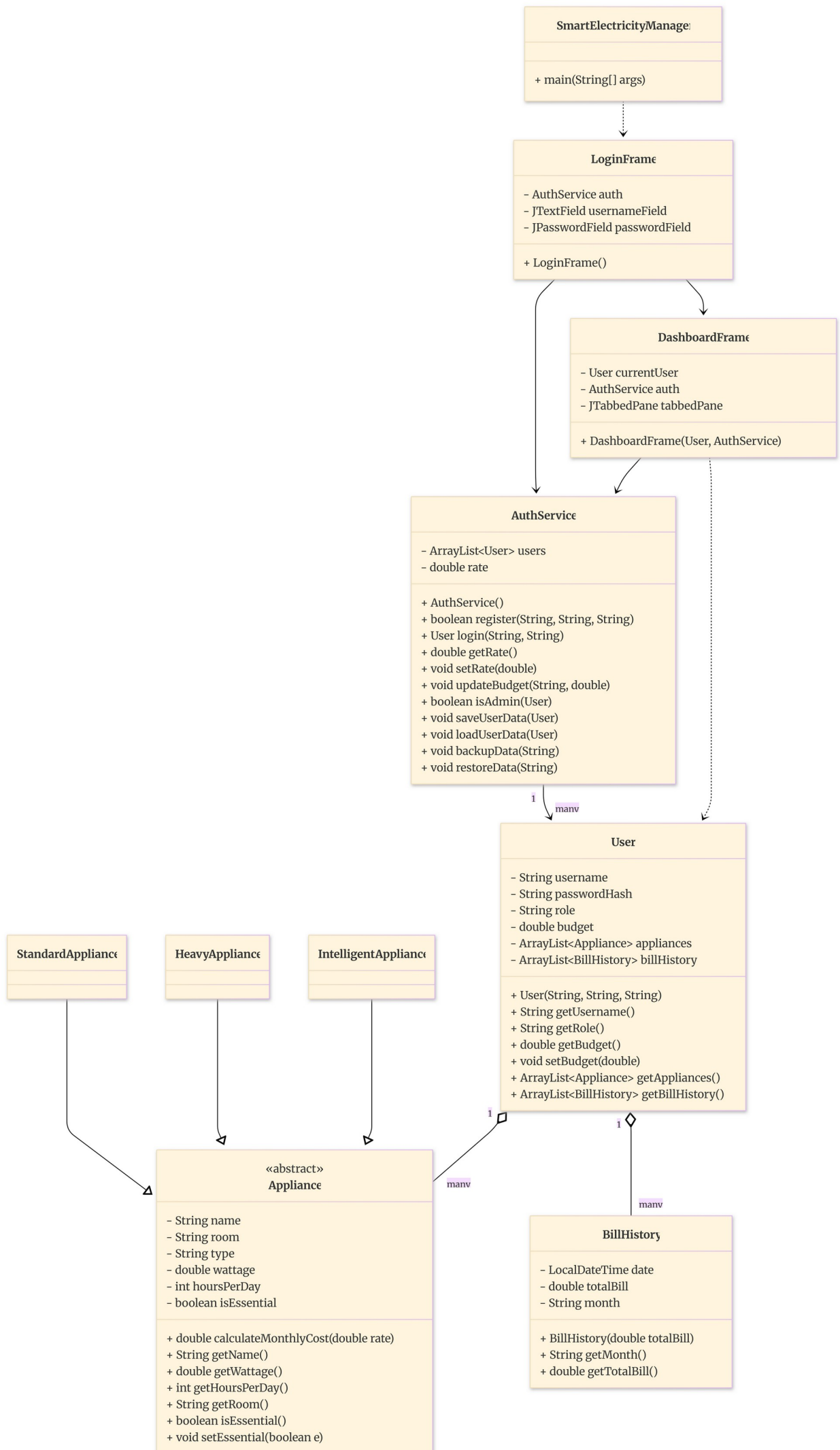
Purpose: Contains the main() method — the starting point of the application. Applies a global dark theme to all Swing components using UIManager, then launches the LoginFrame on the Event Dispatch Thread using SwingUtilities.invokeLater().

Method	Type	Description
main(String[] args)	void	Entry point: calls setDarkTheme() then opens LoginFrame
setDarkTheme()	void	Sets global UIManager properties for dark blue color scheme

```
public class SmartElectricityManager {  
    public SmartElectricityManager() {  
    }  
  
    public static void main(String[] var0) {  
        setDarkTheme();  
        SwingUtilities.invokeLater(() -> (new LoginFrame()).setVisible(true));  
    }  
  
    private static void setDarkTheme() {  
        try {  
            UIManager.put("Panel.background", new Color(18, 25, 45));  
            UIManager.put("Frame.background", new Color(18, 25, 45));  
            UIManager.put("OptionPane.background", new Color(18, 25, 45));  
            UIManager.put("OptionPane.messageForeground", Color.WHITE);  
            UIManager.put("Button.background", new Color(70, 130, 200));  
            UIManager.put("Button.foreground", Color.WHITE);  
            UIManager.put("Button.font", new Font("Segoe UI", 1, 13));  
            UIManager.put("Button.select", new Color(40, 100, 160));  
            UIManager.put("TextField.background", new Color(30, 40, 60));  
            UIManager.put("TextField.foreground", Color.WHITE);  
            UIManager.put("TextField.caretColor", Color.WHITE);  
            UIManager.put("TextArea.background", new Color(30, 40, 60));  
            UIManager.put("TextArea.foreground", Color.WHITE);  
            UIManager.put("PasswordField.background", new Color(30, 40, 60));  
            UIManager.put("PasswordField.foreground", Color.WHITE);  
            UIManager.put("ComboBox.background", new Color(40, 55, 85));  
            UIManager.put("ComboBox.foreground", Color.WHITE);  
            UIManager.put("Label.foreground", Color.WHITE);  
            UIManager.put("TabbedPane.background", new Color(18, 25, 45));  
            UIManager.put("TabbedPane.foreground", Color.WHITE);  
            UIManager.put("TitledBorder.titleColor", new Color(100, 180, 250));  
            UIManager.put("TitledBorder.border", new LineBorder(new Color(60, 90, 130)));  
            UIManager.put("ScrollBar.background", new Color(25, 35, 55));  
            UIManager.put("ScrollBar.foreground", new Color(60, 100, 160));  
        } catch (Exception var1) {  
        }  
    }  
}
```

3. Class Diagram:

The UML class diagram below shows all 10 classes, their key attributes and methods and the relationships between them.



4. GUI Class Mapping:

This section explains which class handles which GUI screen, how button clicks call class methods and how data flows from the GUI to the backend classes.

GUI Screen / Tab	Handled by Classes	Key Interaction
Logine screen	Login Frame AuthService	Login button → auth.login() → opens DashboardFrame Register button → auth.register()
Manual Mode Tab	DashboardFrame AuthService Appliance subclasses	Add button → creates Appliance object → auth.saveUserData() Remove button → removes from user.getAppliances()
Target Mode Tab	DashboardFrame AuthService User	Set Budget button → auth.updateBudget() Optimize button → loops Appliance.calculateMonthlyCost()
Analytics Tab	DashboardFrame Appliance subclasses	Refresh button → updateAnalytics() → calls calculateMonthlyCost() per appliance
Bill History Tab	DashboardFrame BillHistory AuthService	Save Month button → new BillHistory(total) → user.getBillHistory().add() → save
Tips Tab	DashboardFrame	Static content, no backend interaction
Admin Tab	DashboardFrame AuthService	Update Rate button → auth.setRate() → saved to rate.txt
Settings Tab	DashboardFrame AuthService	Backup → auth.backupData() Restore → auth.restoreData() PDF → generateFullPDF() Clear → user.getAppliances().clear()

5. Core Functional Logic Description:

a) Add Appliance:

Location: DashboardFrame.createManualPanel()

- User fills: name, wattage, hours/day, room, selects type (Standard/Heavy/Intelligent)
- Based on type string, creates: new HeavyAppliance() or new IntelligentAppliance() or new StandardAppliance()
- Calls a.setEssential(essentialCheck.isSelected())
- Adds to currentUser.getAppliances() (ArrayList in User object)
- Calls auth.saveUserData(currentUser) - writes to disk
- Calls a.calculateMonthlyCost(auth.getRate()) - shows result in resultArea

- Calls `refreshApplianceList()` and `refreshAllTabs()`

b) Calculate Monthly Cost:

Location: `Appliance.calculateMonthlyCost(rate)`

- **StandardAppliance:** $(\text{wattage} \times \text{hoursPerDay} / 1000.0) \times 30 \times \text{rate}$
- **HeavyAppliance:** $(\text{wattage} \times \text{hoursPerDay} \times 1.15 / 1000.0) \times 30 \times \text{rate}$
- **IntelligentAppliance:** $(\text{wattage} \times \text{hoursPerDay} \times 0.85 / 1000.0) \times 30 \times \text{rate}$
- **Called from:** `DashboardFrame` for display, Analytics panel, Target Mode, PDF generation

c) Save / Load Data (File Handling):

Location: `AuthService.saveUserData()` / `loadUserData()`

- `saveUserData()`: opens `BufferedWriter` to `username_appliances.txt`
- Writes each `Appliance` as `type,name,wattage,hoursPerDay,room,isEssential`
- Also writes bill history to `username_history.txt`: `month,totalBill`
- `loadUserData()`: reads lines, splits by comma, creates matching `Appliance` subclass
- Called on login (load) and after every data change (save)

d) Budget Optimization:

Location: `DashboardFrame.createTargetPanel()` → `optimizeBtn`

- Loops all appliances: `total += a.calculateMonthlyCost(rate)`
- Compares total with `currentUser.getBudget()`
- If over budget: shows over-amount and specific suggestions
- If within budget: shows remaining amount and congratulates user

e) PDF Report Generator:

Location: `DashboardFrame.generateFullPDF(filePath)`

- Creates `iText Document` with `PageSize.A4`
- Adds title paragraph, user info, date, rate, budget
- Creates a `PdfPTable` with 7 columns for appliance data
- Loops all appliances: calls `calculateMonthlyCost()` for each row
- Adds total bill as bold right-aligned paragraph
- If bill history exists: adds second `PdfPTable` with 2 columns
- Closes document → file saved to user-chosen path

f) Authentication:

Location: `AuthService.login()` / `register()`

- `hashPassword()` uses `MessageDigest SHA-256` to convert password to hex string
- `register()`: checks if username exists in list → adds new `User` if not

- login(): retrieves User by username → compares stored hash with hashPassword(inputPwd)
- Returns User object on match, null on failure
- LoginFrame shows error label if null returned

6. OOP Concepts Demonstration:

a) Encapsulation:

All data fields in User, Appliance, BillHistory, and AuthService are declared private. They are only accessed or modified through public getter and setter methods. No external class can directly touch the data.

- **User:** private username, passwordHash, role, budget, appliances, billHistory
- **Appliance:** protected name, wattage, hoursPerDay, room, type, isEssential
- **BillHistory:** private date, totalBill, month
- **AuthService:** private users ArrayList, rate, DATA_DIR
- **Access is only through:** getUsername(), setBudget(), getAppliances(), etc.

b) Inheritance:

Three appliance types — StandardAppliance, HeavyAppliance and IntelligentAppliance all inherit from the abstract class Appliance. They reuse all shared attributes (name, wattage, room, etc.) and only override the calculateMonthlyCost() method.

- Appliance is the parent (abstract) class
- StandardAppliance extends Appliance
- HeavyAppliance extends Appliance
- IntelligentAppliance extends Appliance
- All subclasses inherit: name, wattage, hoursPerDay, room, type, isEssential
- All subclasses inherit: getName(), getWattage(), setEssential() etc.

c) Polymorphism:

The same method calculateMonthlyCost(rate) is called on all three appliance types but each one executes its own version of the formula. This is method overriding the core of runtime polymorphism.

- StandardAppliance.calculateMonthlyCost() standard formula (x 1.0)
- HeavyAppliance.calculateMonthlyCost() adds 15% extra (x 1.15)
- IntelligentAppliance.calculateMonthlyCost() reduces 15% (x 0.85)
- Called uniformly in: Analytics, Target Mode, PDF generation loops

- Java decides which version to call at runtime based on actual object type

d) Abstraction:

The Appliance class is declared abstract. This means it cannot be instantiated directly you cannot write 'new Appliance(...)'. It defines the structure and forces all subclasses to implement calculateMonthlyCost(). The GUI does not need to know which type of appliance it is dealing with it just calls the method.

- abstract class Appliance cannot be used directly with 'new'
- abstract double calculateMonthlyCost(double rate) must be overridden
- The GUI only uses: a.getName(), a.calculateMonthlyCost() no type checking needed
- Hides the internal formula from the GUI layer completely

e) Separation of GUI and Logic:

The GUI classes (LoginFrame, DashboardFrame) never calculate electricity costs or handle file reading/writing themselves. All business logic is in AuthService and Appliance subclasses. The GUI only calls methods and displays results.

- DashboardFrame does NOT calculate bills — it calls a.calculateMonthlyCost().
- LoginFrame does NOT hash passwords — it calls auth.login().
- DashboardFrame does NOT read/write files — it calls auth.saveUserData().
- AuthService handles all data, files, and authentication logic.
- This separation makes the code maintainable and testable.

f) Class Interaction:

The classes work together through well-defined relationships. AuthService manages Users. User holds a list of Appliances and BillHistory objects. DashboardFrame uses both AuthService and User to display and update data.

- SmartElectricityManager creates LoginFrame
- LoginFrame creates AuthService (one instance shared throughout)
- LoginFrame creates DashboardFrame (passes User and AuthService)
- DashboardFrame uses User.getAppliances() to display list
- DashboardFrame uses AuthService.saveUserData() to persist changes
- User has-a ArrayList and ArrayList

7. Graphical User Interface:

Login Page

Smart Electricity Bill Manager

SMART ELECTRICITY BILL MAN...

Pakistan – Control Your Power, Save Your Money

USERNAME

Nigah Fatima

PASSWORD

.....

LOGIN

REGISTER

Dashboard

Dashboard - Nigah Fatima

Welcome, Nigah Fatima00:00:35Rate: Rs35.00/kWhBudget: Rs10000.00

Physical ModeTarget ModeAnalyticsBill HistoryTips & AdviceSettings

+ ADD NEW APPLIANCE

Appliance Name:

Wattage (W):

Hours per Day:

Room:

Type:

Standard

Essential Appliance

+ ADD & CALCULATE

YOUR APPLIANCES (click to select)

REFRESH LIST

REMOVE SELECTED

Logout

Manual Mode

Here you can add, refresh and remove an appliance. Here the bill for each appliance will be calculated automatically.

Welcome, Nigah Fatima

00:01:06 < Rate: Rs35.00/kWh Budget: Rs10000.00

Manual Mode Target Mode Analytics Bill History Tips & Advice Settings

+ ADD NEW APPLIANCE

Appliance Name:

Wattage (W):

Hours per Day:

Room:

Type:

☒ Essential Appliance

+ ADD & CALCULATE

YOUR APPLIANCES (click to select)

Fan (3) - Standard - 40W x 5h = ₹210.00/m

Light (2) - Standard - 15W x 8h = ₹126.00/m

Refrigerator (1) - Standard - 180W x 20h = ₹3780.00/m

AC (1) - Heavy - 800W x 4h = ₹3864.00/m

Microwave (1) - Standard - 600W x 4h = ₹2520.00/m

REFRESH LIST

REMOVE SELECTED

Logout

Target Mode

Here you can set your budget first.

Welcome, Nigah Fatima

00:02:57 < Rate: Rs35.00/kWh Budget: Rs10000.00

Manual Mode Target Mode Analytics Bill History Tips & Advice Settings

SET YOUR MONTHLY BUDGET

Budget (Rs):

SET BUDGET

OPTIMIZE USAGE

Logout

The user set a budget of Rs.10,000 but the current bill is Rs.10,500 so it is Rs.500 over budget and the system is showing recommendations to reduce electricity usage.

Welcome, Nigah Fatima

00:04:23Rate: Rs35.00/kWhBudget: Rs10000.00

Manual ModeTarget ModeAnalyticsBill HistoryTips & AdviceSettings

SET YOUR MONTHLY BUDGET

Budget (Rs):10000.0

SET BUDGETOPTIMIZE USAGE

===== OPTIMIZATION REPORT =====

Current Bill: Rs10500.00
Budget: Rs10000.00
Over budget: Rs500.00

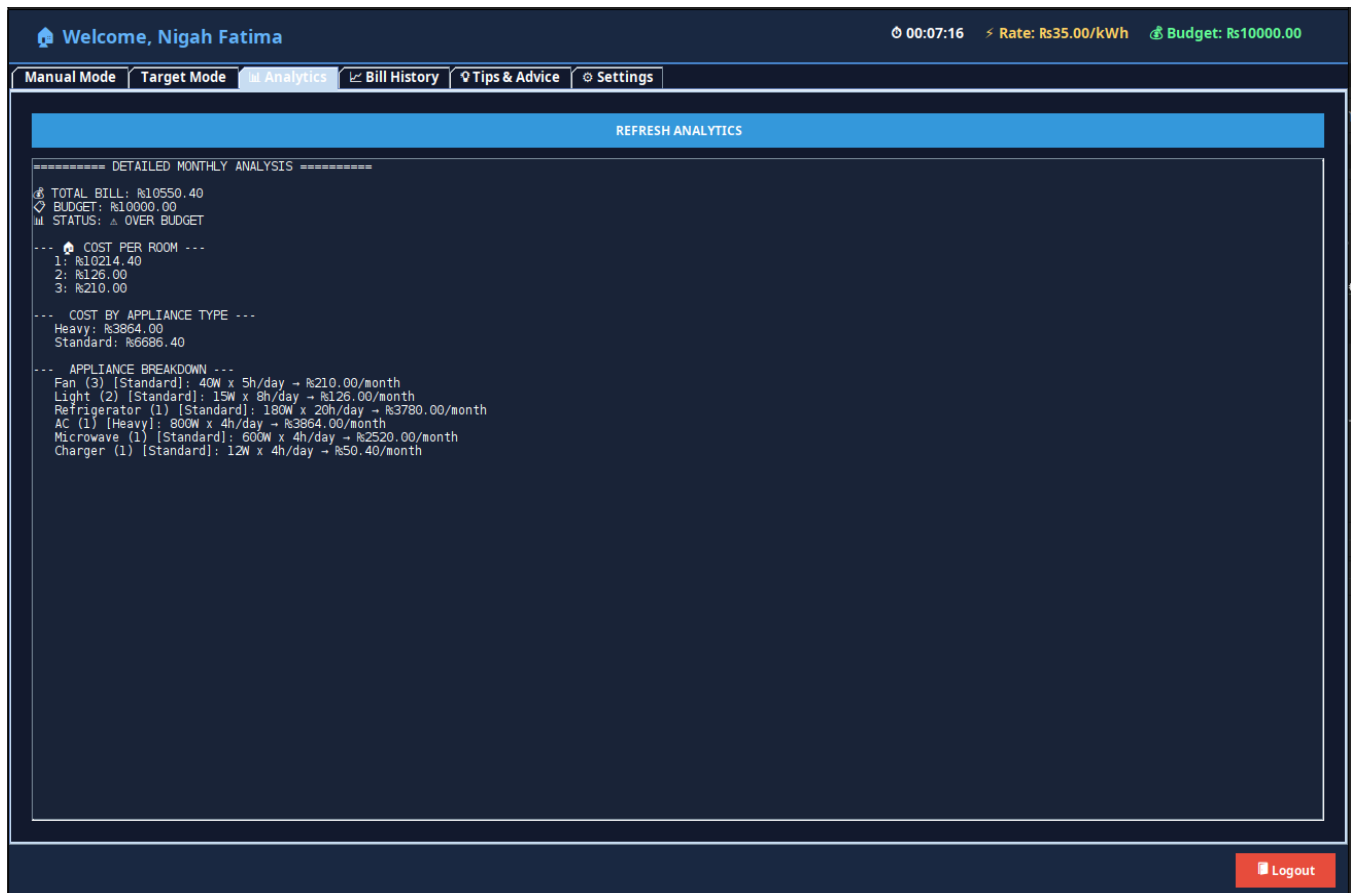
RECOMMENDATIONS:

- Reduce AC/Heater usage by 2-4 hours daily
- Replace bulbs with LEDs (save up to 75%)
- Unplug idle devices (standby power 5-10%)
- Use fan instead of AC when possible
- Shift heavy tasks to off-peak hours (after 10 PM)

Logout

Analytics Tab

It shows a detailed monthly breakdown the total bill is Rs.10,550.40 which is over the Rs.10,000 budget and it breaks down the cost by room, by appliance type (Heavy vs Standard) and shows each appliance individually with its wattage, daily hours and monthly cost.



Bill History Tab

The user clicked “Save Current Month” and it saved May 2026 bill as Rs.10,500.00 this record is now stored in the BillHistory object and displayed in the history list.



Tips and Advice Tab

It displays Pakistan-specific energy saving tips across categories like Lighting, Cooling, Refrigerator and Standby Power. It is purely static content, no buttons or method calls involved.



Settings Tab

It provides 5 actions update electricity rate, backup data, restore data, export a full PDF report and clear all appliances.

